



# 从“可用”到“卓越”：RAG系统的高级优化工程

---

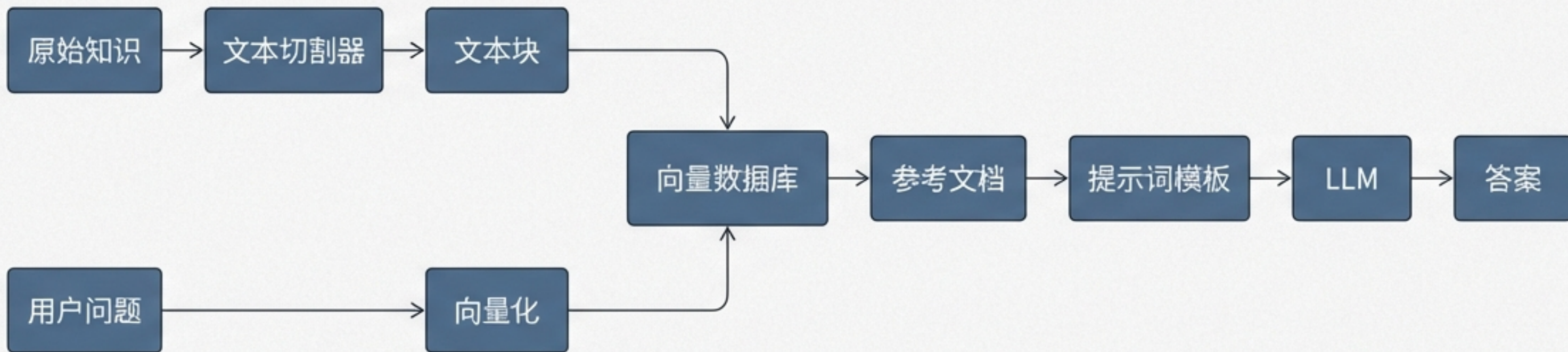
一份面向构建者的深度实践指南



# RAG的核心挑战：算法问题，更是工程问题

一个基础的RAG系统是起点，而非终点。其性能瓶颈往往出现在数据处理、检索精度和生成质量的工程细节中。

基础RAG流程 (“Vanilla” RAG Pipeline)





# 优化蓝图：三大核心模块的系统性升级

## 1. 输入与数据优化

从源头提升质量：优化输入问题和知识元数据。

基础RAG流程 (“Vanilla” RAG Pipeline)

## 3. 生成优化

综合提炼答案：有效利用检索到的上下文生成高质量回复。

## 2. 搜索优化

精准定位信息：实现更高召回率与相关性的检索策略。

原始知识

文本切割器

文本块

向量数据库

参考文档

提示词模板

LLM

答案

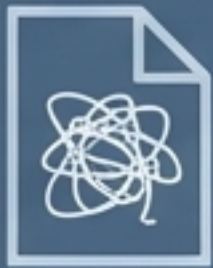
用户问题

向量化



# 输入优化 I：通过改写提问，明确意图

用户的原始提问往往不是最优的检索输入。改写提问是保证“垃圾进，垃圾出”原则不被触发的第一道防线。



## 提问不规范 (Poorly Formed Queries)

**Problem:** 语义模糊、歧义、语法不通、提问不规范。

**Solution:** 使用LLM将问题标准化、清晰化。



## 多轮对话 (Multi-turn Conversations)

**Problem:** 用户提问依赖于上文，存在指代。

**Solution:** 结合历史问答对提问改写，进行指代消解，形成独立的、完整的检索问题。



## 复杂提问 (Complex Queries)

**Problem:** 一个问题包含多个子问题或意图。

**Solution:** 将复杂问题拆解成多个独立的子问题，分别进行RAG，然后组合成最终答案。



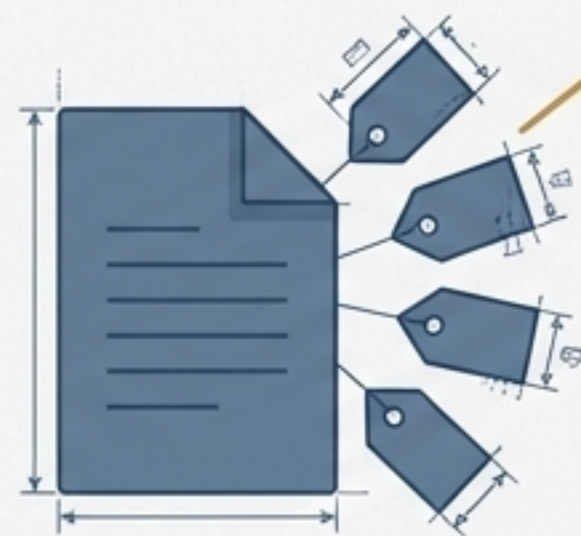
# 输入优化 II：利用元数据，实现智能过滤与排序

**什么是元数据？** 对文档的描述性信息，例如类型、作者、创建日期、来源等。

## 检索后重排 (Post-retrieval Re-ranking)

Action: 在获得初步检索结果后，根据元数据对文档列表进行重新排序。

示例例：确保来自权威来源的文档排在最前面。



## 检索前过滤 (Pre-retrieval Filtering)

Action: 在向量搜索前，先根据元数据剔除无关文档。

示例：“只检索最近一个月的文档”或“只检索特定来源的报告”。

## 检索中评估相关性 (In-retrieval Relevance Assessment)

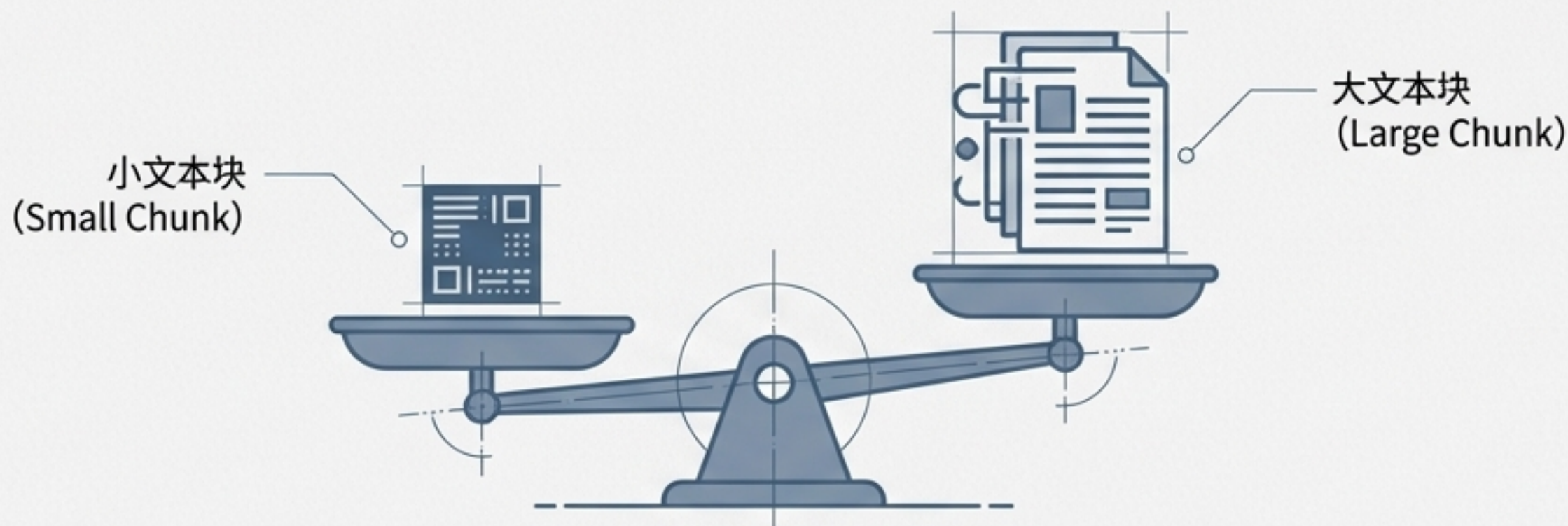
Action: 将元数据作为信号，调整文档的相关性得分。

Example: 对更重要或更新的文档给予更高的权重加成。



# 搜索优化：平衡精度与上下文的艺术

## 检索的核心矛盾 (The Core Conflict in Retrieval)



### 小文本块 (Small Chunks)

- **优点 (Pros)**：语义集中，embedding表征更精确，提升检索匹配度。
- **缺点 (Cons)**：上下文信息不足，LLM无法获得完整的背景知识。

### 大文本块 (Large Chunks)

- **优点 (Pros)**：提供更丰富的上下文，帮助LLM生成更全面、连贯的答案。
- **缺点 (Cons)**：包含噪声信息，可能稀释核心语义，导致检索精度下降。

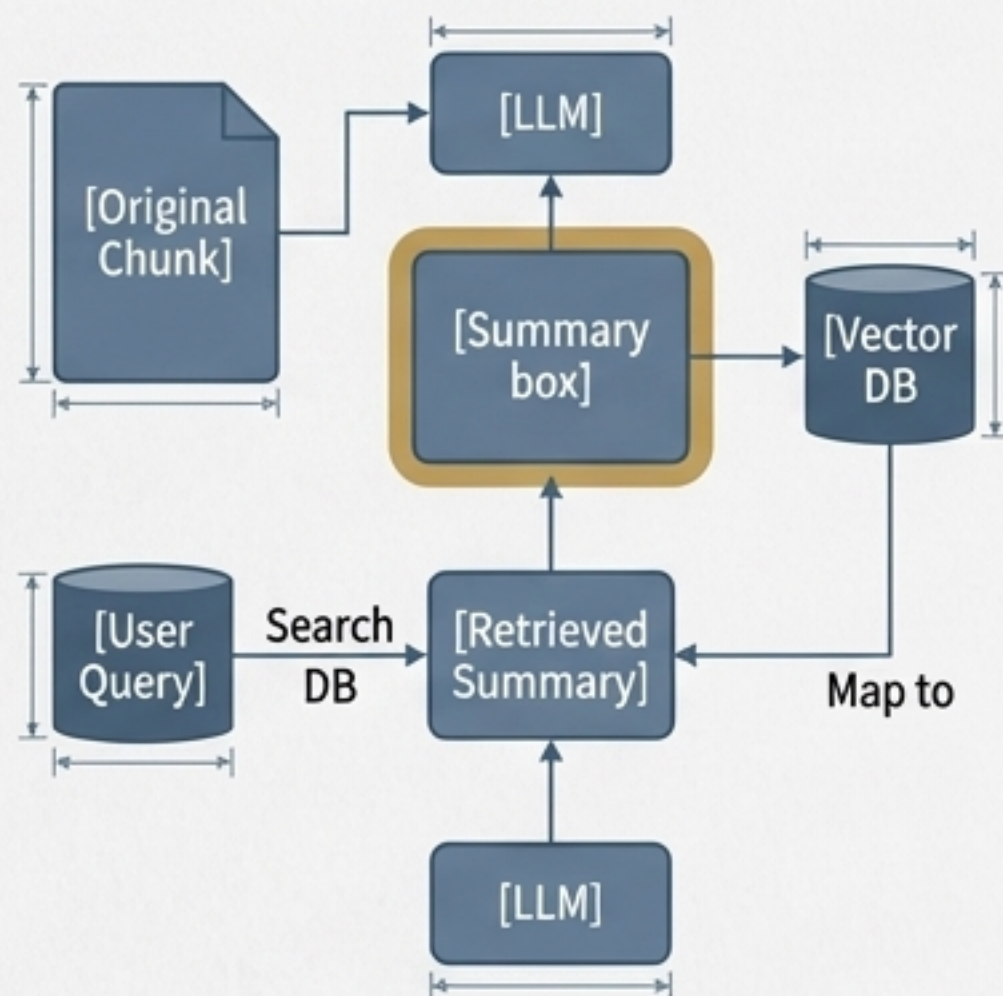
**“Small-to-Big” 策略**：检索时用小文本块，生成时用大文本块。



# Small-to-Big 的三种实现路径

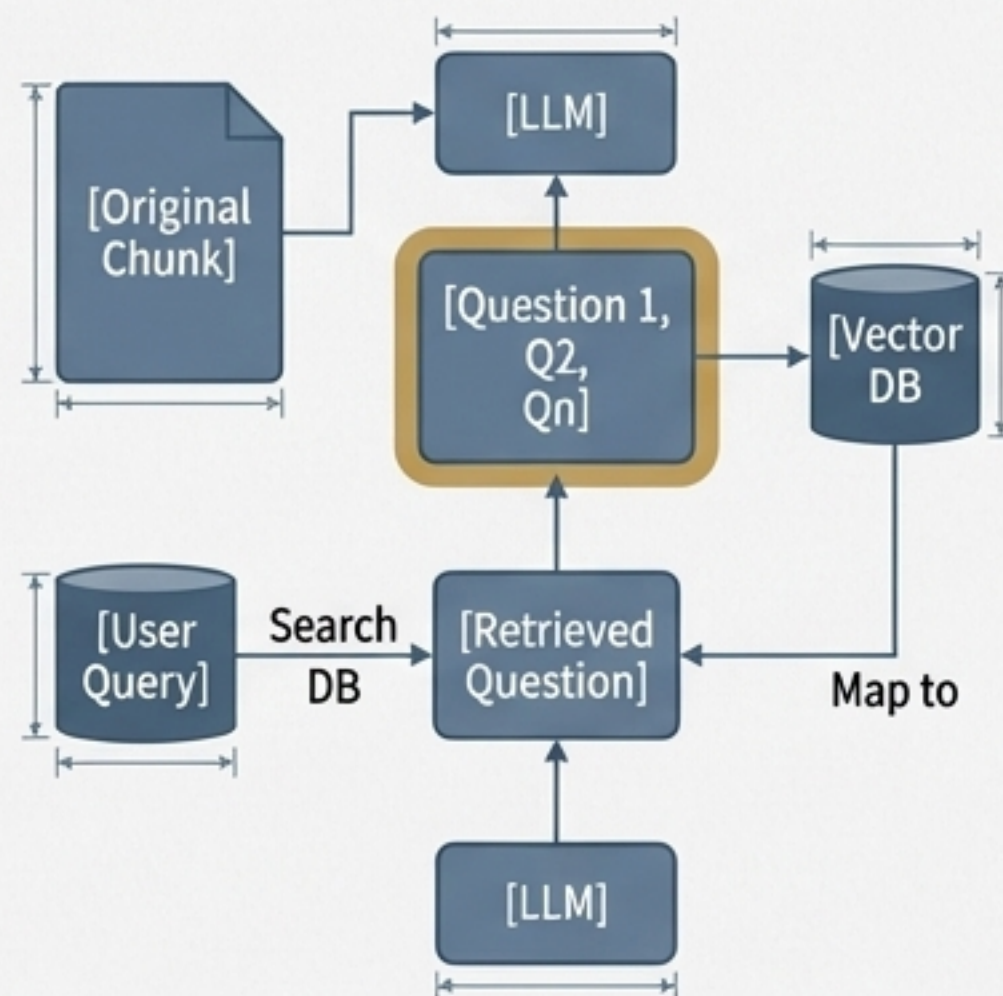
## 1. 摘要检索 (Summary Retrieval)

对每个文本块生成一个高质量摘要，对摘要进行向量检索。检索到摘要后，将对应的原始、完整的文本块提供给LLM。



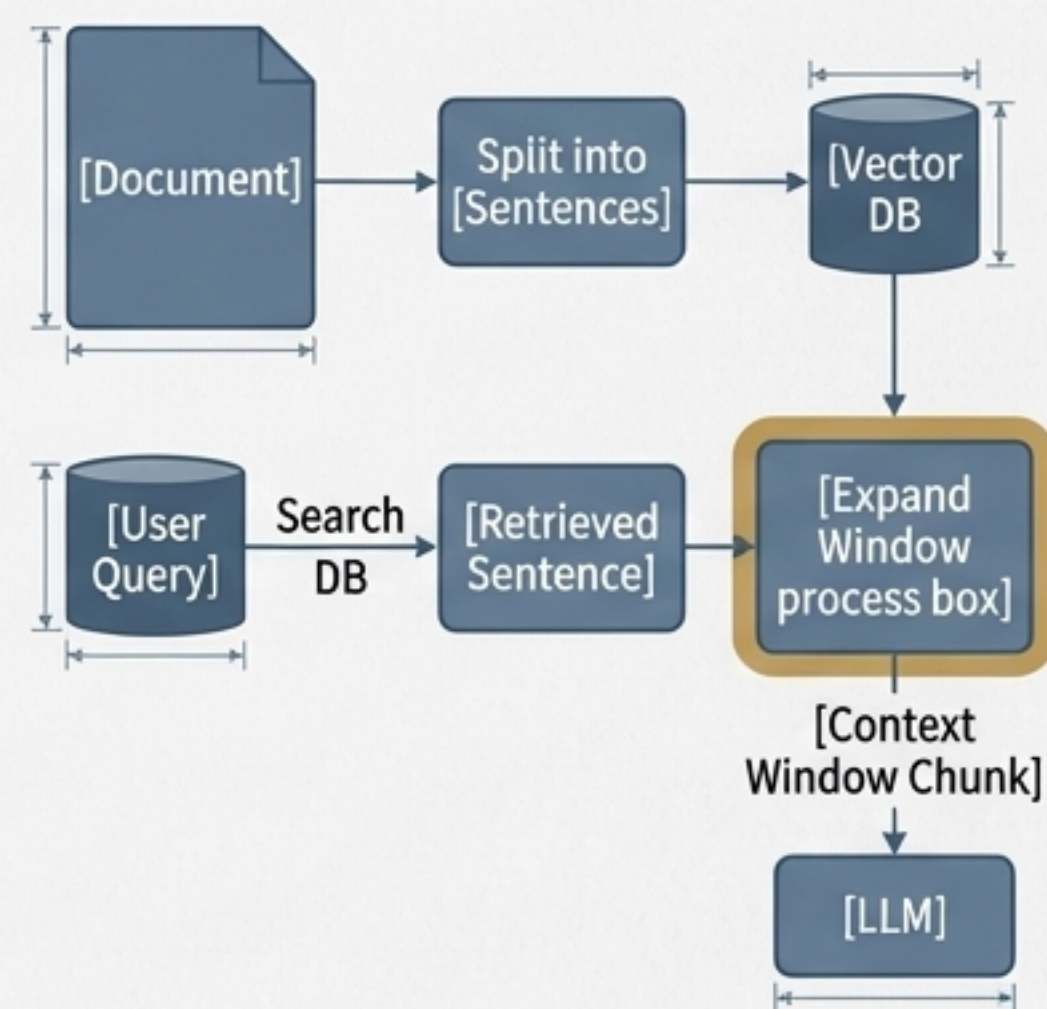
## 2. 子问题检索 (Sub-question Retrieval)

让LLM针对每个文本块生成可能被问到的若干子问题。检索时，将用户问题与这些生成的子问题进行匹配。



## 3. 句子窗口检索 (Sentence Window Retrieval)

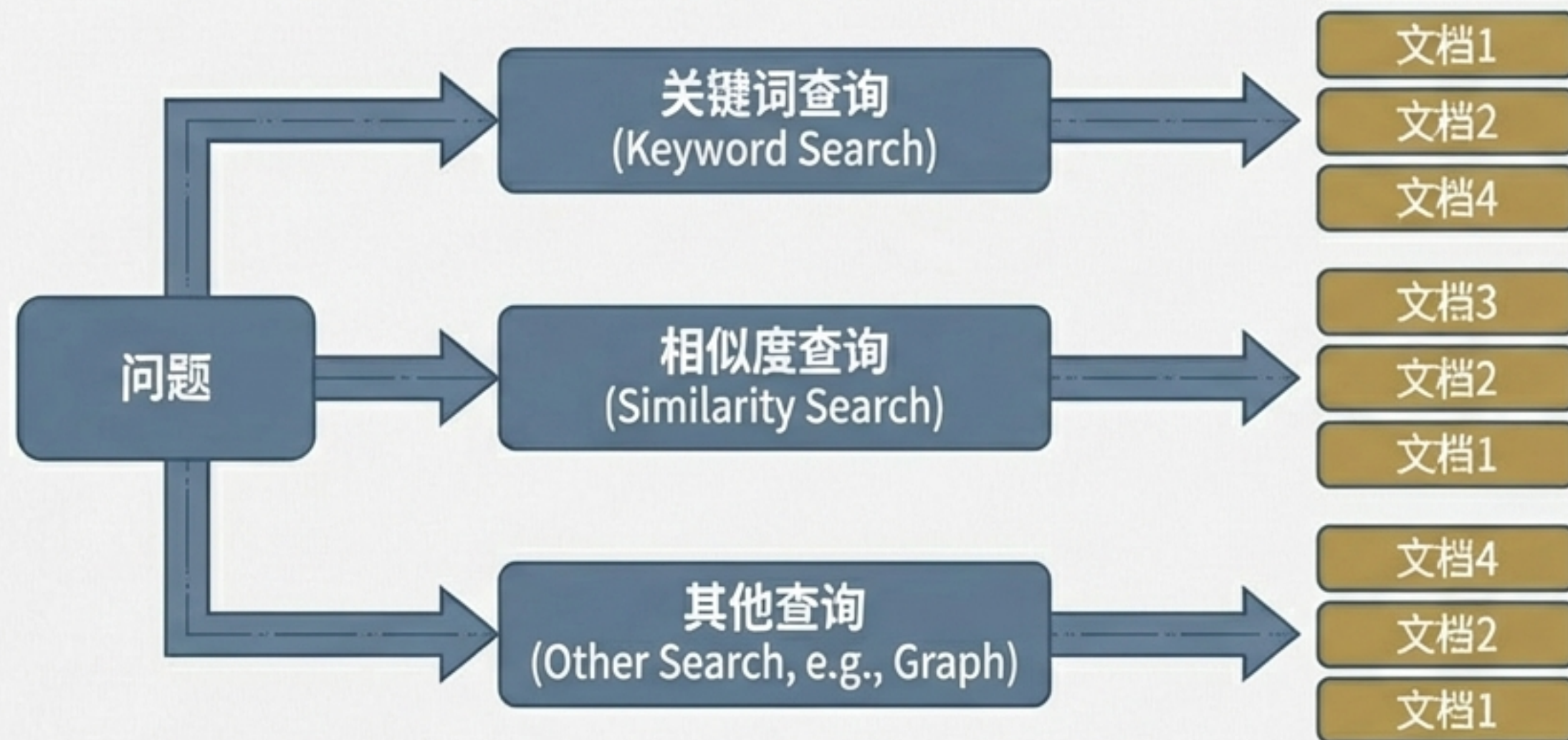
将单个句子作为检索单元。检索到最相关的句子后，将其前后N个句子的“窗口”拼接起来，作为完整的上下文提供给LLM。





# 搜索优化 II：多路召回，避免单点故障

任何单一的检索方法都有其局限性。向量检索擅长语义，但可能丢失关键词；关键词检索反之。多路召回融合多种方法，取长补短。



如何将这些不同的结果列表，融合成一个更优的排序？



# 结果融合：使用倒数排名融合（RRF）算法

RRF是一种简单而高效的无参数融合方法。它不关心不同检索器的原始得分，只关心文档在各自列表中的排名。一个文档的最终得分是其在所有列表中排名的倒数之和。

$$RRF\ Score(d) = \sum \frac{1}{rank_i(d)}$$

| Document | Search 1 Rank   | Search 2 Rank   | Search 3 Rank   | RRF Score |
|----------|-----------------|-----------------|-----------------|-----------|
| 文档1      | 1 (Score: 1.00) | 3 (Score: 0.33) | 3 (Score: 0.33) | 1.67      |
| 文档2      | 2 (Score: 0.50) | 2 (Score: 0.50) | 2 (Score: 0.50) | 1.50      |
| 文档3      | -               | 1 (Score: 1.00) | -               | 1.00      |
| 文档4      | 3 (Score: 0.33) | -               | 1 (Score: 1.00) | 1.33      |

通过RRF，即使在某一检索方法中排名不高但在多个列表中都出现的文档（如文档2），其综合排名也会得到提升。



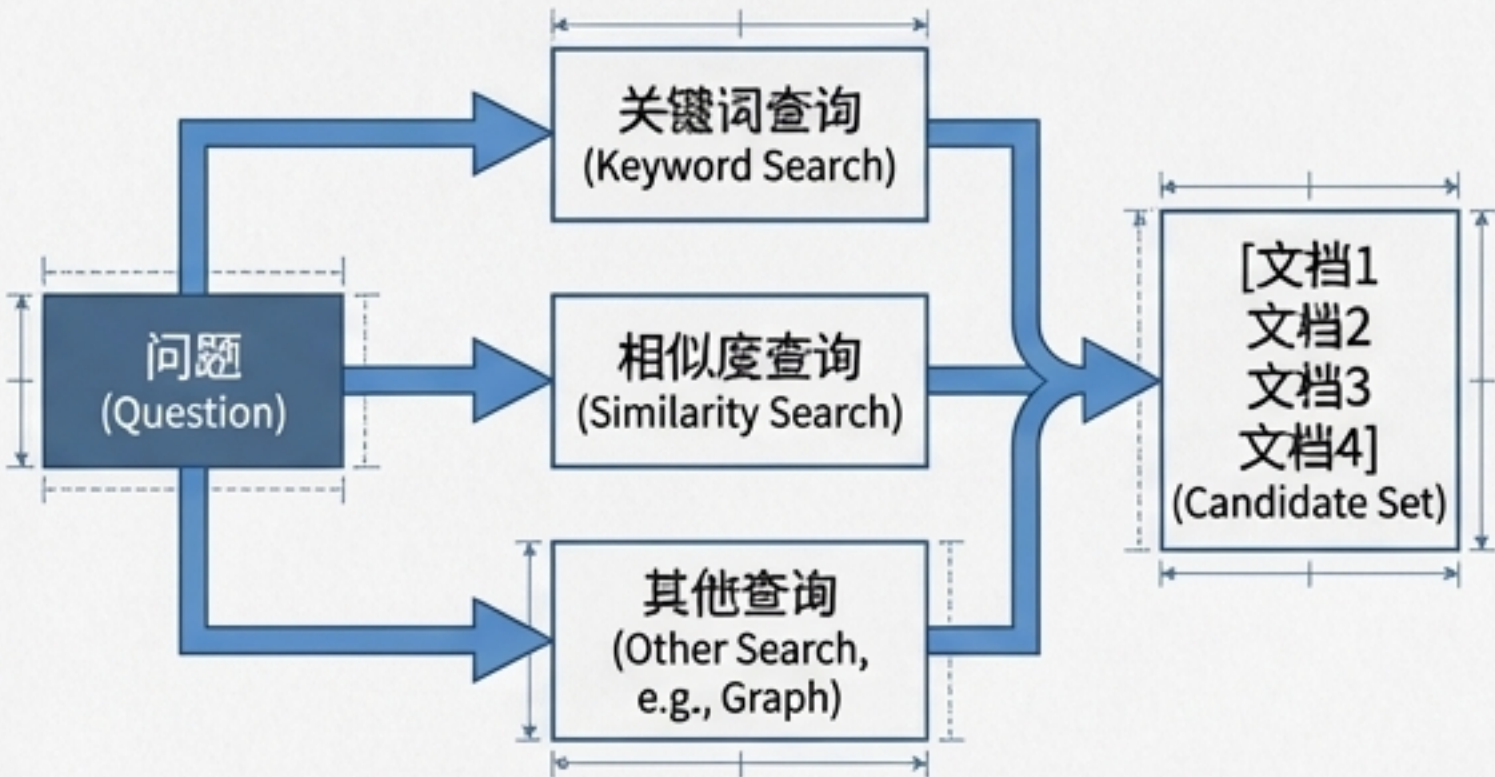
# 搜索优化 III: Re-rank, 从粗筛到精选

Re-rank是一个双层筛选过程，旨在平衡检索速度和精度。

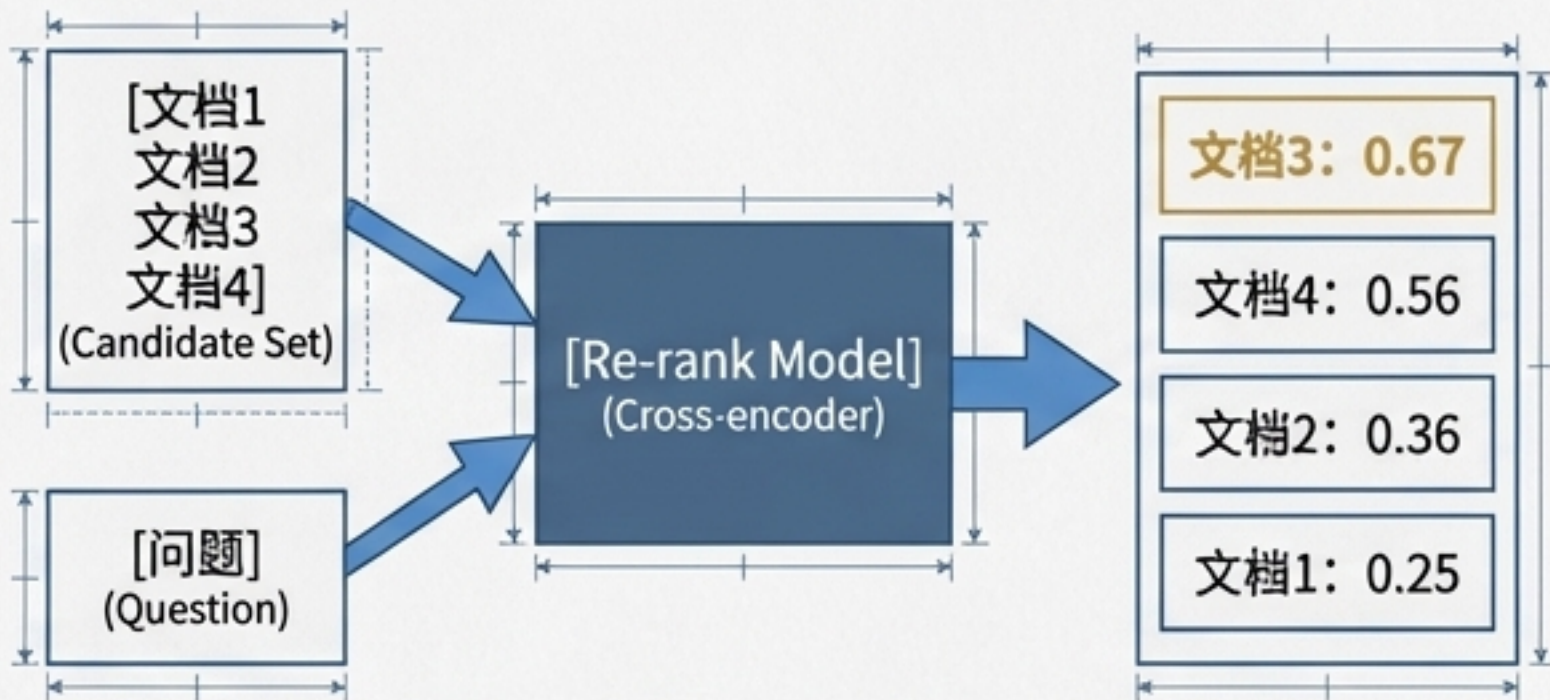
粗筛（Coarse Filtering）

精选（Fine Selection）

使用一种或多种快速的检索方法（如多路召回）从海量文档中召回一个较大的候选集（例如Top 100）。此阶段追求“快”和“全”。



使用一个更强大、更复杂的模型（通常是Cross-encoder）对候选集进行重新打分排序。该模型会同时考虑“问题”和“文档”，进行更精细的相关性判断。此阶段追求“准”。





# 生成优化：当检索到的信息过多时

成功检索到大量相关文档后，新的问题随之而来。



**上下文溢出 (Context Overflow):**  
检索到的文档总长度超过LLM的上下文窗口限制。



**知识不连贯 (Knowledge Discontinuity):**  
来自不同文档的知识块可能缺乏逻辑联系或存在矛盾。



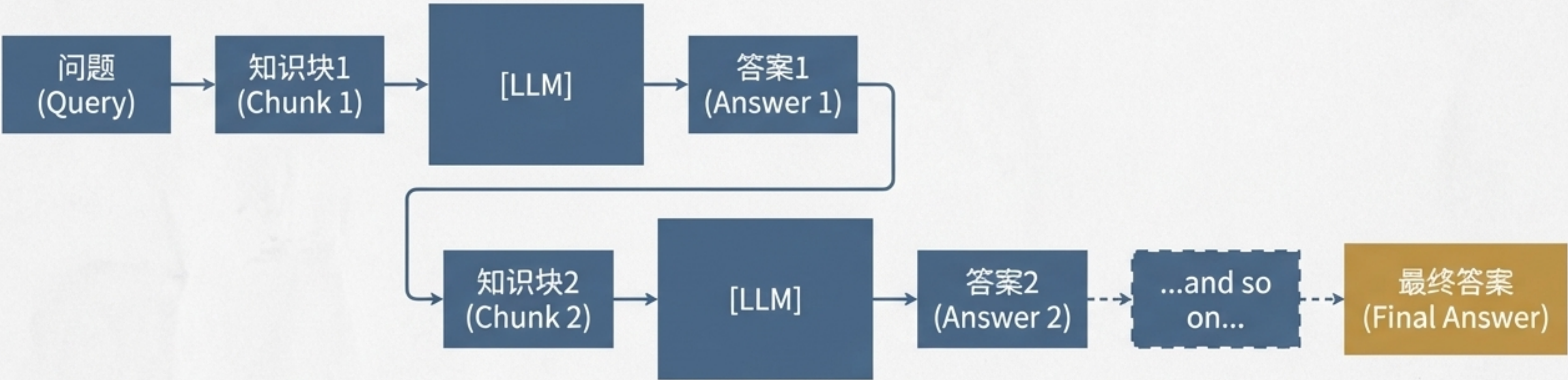
**核心信息淹没 (Key Information Drowning):**  
重要的信息可能被大量次要的细节所淹没，导致LLM抓不住重点。

## 如何让LLM高效、智能地“消化”这些大量的参考文档？



# 生成策略 I: Refine模式，迭代优化答案

Refine方法通过串行处理文档来逐步构建和完善答案。它将每个文档块依次喂给LLM，并要求LLM在前一个答案的基础上进行修正和补充。

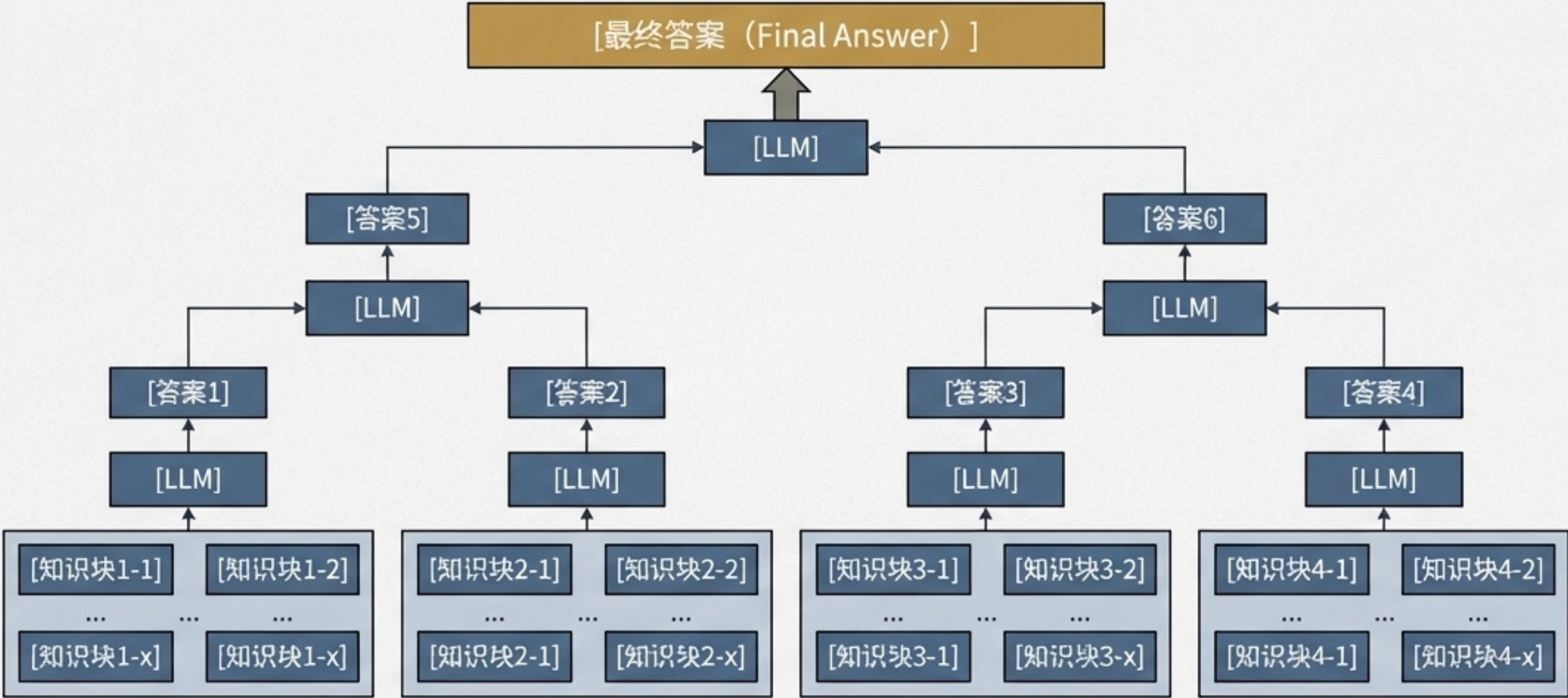


| 最适用场景 (Best for)                  | 缺点 (Downside)                     |
|-----------------------------------|-----------------------------------|
| 需要构建一个高度综合、细节丰富的答案，且文档之间有递进或补充关系。 | 速度较慢，因为LLM调用是串行的；且答案可能受文档处理顺序的影响。 |



# 生成策略 II: Tree-Summarize, 分而治之

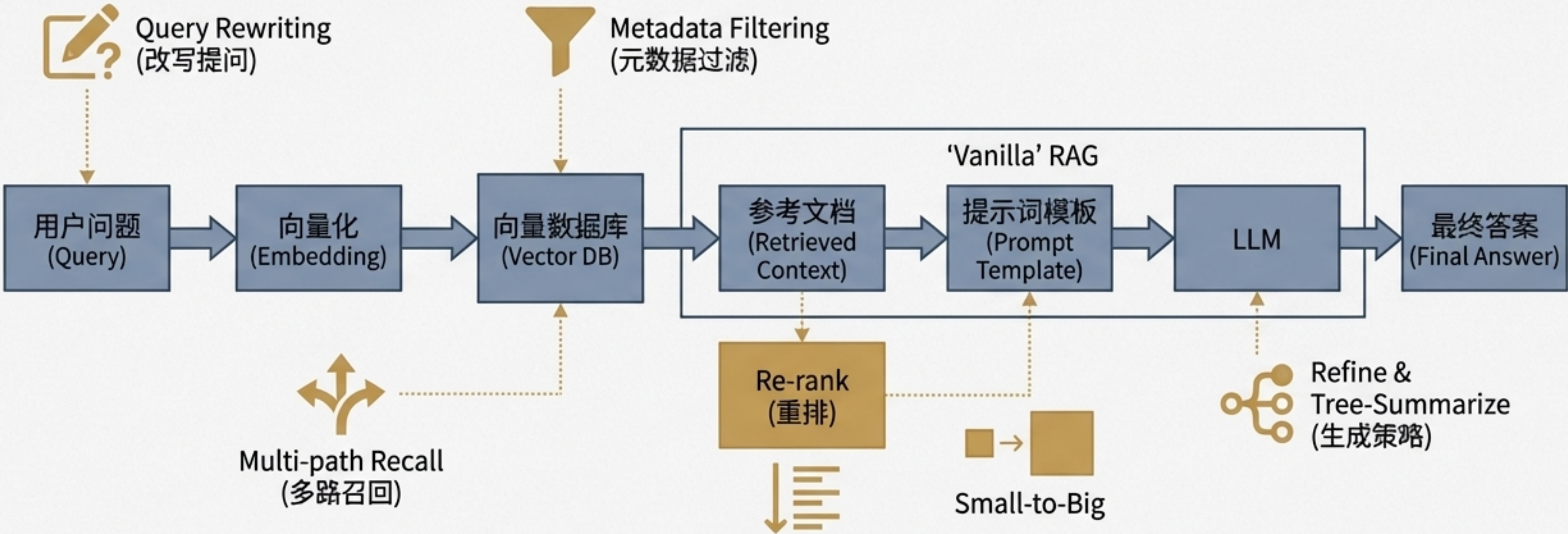
Tree-Summarize采用分层的、树状的总结方法。它首先并行地总结小批量的文档，然后递归地对上一层的总结进行再总结，直到生成最终的全局答案。



| 最适用场景 (Best for)                  | 缺点 (Downside)          |
|-----------------------------------|------------------------|
| 需要处理海量文档，且对响应速度有较高要求。特别适合主题概括类任务。 | 在信息传递过程中，底层文档的细节可能会丢失。 |



# 终极蓝图：构建你的高性能RAG系统



卓越的RAG系统并非单一技术的产物，而是根据具体场景，对各个环节进行模块化、系统性优化的工程结晶。